Mobile Robotics **Fall 2026**

Homework 02

Odometry, Control, and Gazebo

CE/EE 468 ▪ Due October 12, 2026 ▪ Basit Memon ▪ Total **135** pts

---

Submission. Work in the same pairs as Homework 1. Submit *one* PDF per group, plus *one GitHub repository link per student*. All code should be shared as a GitHub repository link.

Learning focus

This assignment is about the gap between what a robot *believes* and what it *does*. You will **find and calibrate odometry error**: faults that `/odom` never reports, and the UMBmark procedure that exposes them. You will **close the loop with a sensor**, tuning P, PD and PID controllers so that robot can travel along a path despite errors.

Homework 3 picks up from here: you will supplement wheel odometry with exteroceptive sensors to localize the robot accurately.

GROUND RULES

Your robot is yours. Every pair simulates a slightly different robot. Your pair ID is your group number on Canvas; your robot's dimensions are in `config/pairs.yaml`. Numbers from another pair's robot will not match your data.

AI tools. Permitted for infrastructure: build errors, shell commands, boilerplate, plotting, reading error messages. Not permitted for the derivations or for the modelling arguments. Where you used a tool, note it in one line at the end of that answer.

SUPPORT

If you're struggling with the setup, don't wait until the last minute. You can reach out to the class TA (Mubashir) for help.

The one condition is that you come early. Support cannot rescue you the night before.

Problem 1 Meet your robot [10 pts]

Maps to CLO 1 | **Bloom's Level** Apply

Context. The Gazebo problems in this homework all run on one simulated differential-drive robot, provided as the ROS 2 package `a2_description`. This problem gets it running on your machine and has you read what is inside it; the next one has you write your first node that drives it.

GIVEN

- **The package** `a2_description` (repository). Its `README.md` explains how to build and run it, lists every topic and frame, and has a troubleshooting table. Read it first.
- **A video:** A. Haber, *ROS2 Jazzy Tutorial: Develop and Control Gazebo Model of Mobile Robot in ROS2 and Python*, https://www.youtube.com/watch?v=campHHAPtyg, shows how to build a differential-drive robot in Gazebo from scratch. **You do not need to rebuild the robot**; watch the video to understand what each file in the package is for. There are more videos on URDF and Gazebo in the ROS document shared with you last time.
- **Your robot:** The package provides different robot constructions – a reference robot `REF` and twelve pairs of slightly different robots. Each pair's robot has slightly different wheel radii and track width, while the reference robot has the standard dimensions. The numbers are in `config/pairs.yaml` under your pair ID, the `actual` block gives the left and right wheel radii r_L, r_R and the track width b that **will be used by Gazebo** to simulate the robot's behavior. Write $\bar{r} = (r_L + r_R)/2$.
- **Ground truth:** the topic `/ground_truth/odom` gives the robot's *true* pose, read from the Gazebo physics engine. It is not being computed from integrating wheel displacements. A real robot has no such thing; treat it as a perfect external tracking system.

TASKS

(a) **Run it.**

(i) Build the package in a workspace and launch the simulation for your pair:

```
ros2 launch a2_description sim.launch.py pair:=P03    # your pair ID
```

The terminal prints the physical dimensions and the values given to the diff-drive plugin. Copy both lines into your report.

(ii) Drive the robot with `teleop_twist_keyboard` (see the README).

(iii) With the simulation running, use `ros2 node list`, `ros2 topic list`, `ros2 topic hz` and `ros2 topic echo` to find, for each topic in the README's table: which node publishes it, which nodes subscribe to it, and how often it is published. Include `rqt_graph`'s picture of the running system.

(iv) Launch `display.launch.py` for your pair. Drag each wheel slider and watch its frame in RViz.

(v) Generate the frame diagram with `ros2 run tf2_tools view_frames` while the simulation runs.

CHECKPOINT

- A positive linear command drives the robot *forward* (along $+x$ of `base_link`); a positive angular command turns it *counter-clockwise* seen from above.
- In RViz (fixed frame `world`) you see the robot, red LiDAR points on the walls, blue arrows from `/odom` and green arrows from `/ground_truth/odom`.
- The frame diagram has `world` at the top, with two branches: `odom` → `base_link` → the four child links, and `base_link_gt`.

If any of these fails, stop and fix it; the README's troubleshooting table covers most cases. Everything after this depends on it.

Read the model. Answer from the files, not from memory. To see exactly what Gazebo receives for your robot, expand the xacro into plain URDF (command in the README, §6) and read that too.

TASKS

(b) **Map the package to the video.** For each of the six files below, give the time (*mm:ss*) in the video where the equivalent content is built, and one sentence on what the file does: `properties.xacro`, `a2_bot.urdf.xacro`, `a2_bot.gazebo.xacro`, `bridge.yaml`, `a2_arena.sdf`, `sim.launch.py`.

(c) **A joint is a transform.** From the expanded URDF for your pair, write the homogeneous transform ${}^{B}\boldsymbol{T}_{L}$ from `base_link` to `left_wheel_link` that the left wheel joint's `<origin>` encodes, with numbers. Use it to find the left wheel's ground-contact point in `base_link` coordinates.

Problem 2 Drive a square, open loop [30 pts]

Maps to CLO 3 | **Bloom's Level** Analyze

Context. "Open loop" or "Feedforward" means the robot is given a command and it does not use feedback to adjust its behavior: drive straight for a fixed time, turn for a fixed time, repeat. You use the robot from Problem 1.

TASKS

(a) **Write a node** `open_loop_square` (Python, `rclpy`) in your repository. With parameters[a] for the side length L, speed v, turn rate ω and direction (counter-clockwise or clockwise), it should publish on `/cmd_vel`, which contains (v, ω). You can move in a square by a sequence of straight line and rotating motions, executed for fixed durations.

Time everything with the node's clock, `self.get_clock()`, and run the node with `use_sim_time:=true`. While it runs, log the pose from `/odom` and from `/ground_truth/odom` to a CSV file.

[a] The official ROS2 guide to parameters: https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Using-Parameters-In-A-Class-Python.html

TASKS

(b) **Run** the square three times with `pair:=REF` and three times with your own pair ID, relaunching the simulation before each run. Use $L = 2\,\text{m}$, $v = 0.2\,\text{m/s}$, $\omega = 0.5\,\text{rad/s}$, counter-clockwise, starting at the origin.

For each run, report the final position (distance from the start) and heading, according to `/odom` and according to `/ground_truth/odom`.

The difference between these two topics is that `/ground_truth/odom` is the true pose of the robot, observed directly from the simulation, while `/odom` is an estimate of the pose based on the wheel encoder data. Problem 3 comes back to this.

(c) **Plot** one run of each robot: the commanded square, the `/odom` path and the true path, on the same axes with equal scaling.

(d) Your commanded (v, ω) is sent to the **Gazebo differential-drive plugin**. The plugin accepts a single wheel radius r and a track width b (`a2_bot.gazebo.xacro`). Your physical robot has two different wheel radii; **Which value is being given to the plugin?** (Read `properties.xacro` and your launch output.)

(e) **Explain.**

(i) Why must the node use simulation time rather than your computer's clock? What would go wrong with the computer's clock if the simulation ran at half of real time?

(ii) Even the `REF` robot, whose dimensions the plugin knows exactly, does not close the square perfectly. Name two causes. Change one setting of your node to test one of them, and report the result.

(f) Your own robot's wheels are not quite equal. Suppose you command a straight line, $\omega = 0$, and drive a distance d. Derive, in terms of r_L, r_R, b and d, the *true* heading change $\Delta\theta$ and the sideways offset Δy at the end. Use a small-angle approximation for Δy. Evaluate both for your robot with $d = 2\,\text{m}$. Which way does the true path curve? Do you observe this behavior in your runs?

(g) **How does the Gazebo differential-drive plugin turn your command into wheel speeds?** The package never writes the differential-drive kinematic model down anywhere. The model lives in Gazebo's own source code, for the version this course uses: `gz-sim/src/systems/diff_drive/DiffDrive.cc` (branch `gz-sim8`). Find the lines that turn a commanded (v, ω) from `/cmd_vel` into left and right wheel angular velocities. Quote them, and show they are the kinematic model from class, solved for the wheel speeds.

Problem 3 Odometry that lies [15 pts]

Maps to CLO 1, 3 | **Bloom's Level** Analyze

Context. Problem 2 was about the *command* side of the diff-drive plugin: how a (v, ω) you publish becomes wheel motion. This problem is about its *estimate* side: how wheel motion becomes `/odom`. The plugin computes `/odom` from the wheel joints using *its own* wheel radius and track width. This problem and the next find out how odometry goes wrong when those

numbers do not match the robot, and why nothing warns you when it does.

TASKS

(a) **How does the plugin compute `/odom`?** In Problem 2(g) you found the plugin's command side. Its odometry side lives in `gz-math/src/DiffDriveOdometry.cc` (branch `gz-math7`).

(i) Find the lines that turn the change in wheel joint angles into the increments Δs and $\Delta\theta$ used for `/odom`. Quote them and match them to the model from class.

(ii) The pose is then updated by one of two integration schemes, chosen by the size of $\Delta\theta$. Which schemes are they? Why does the code switch schemes when $\Delta\theta$ is tiny?

(b) For the same case of the robot moving on a straight line, derive, in terms of r_L, r_R, b and d, what `/odom` will report for $\Delta\theta$ and Δy.

(c) Start from a fresh launch (robot at the origin, facing $+x$). Drive straight for about 15 s:

```
ros2 topic pub --rate 20 /cmd_vel geometry_msgs/msg/Twist "{linear: {x:
    0.2}}"
# Ctrl-C after ~15 s, then stop the robot:
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{}"
```

Record the final pose from `/ground_truth/odom` and from `/odom`. Take d as the true distance travelled. Do three runs. Tabulate predicted and measured $\Delta\theta$ and Δy, and compare. Does this account for what you saw in Problem 2?

Problem 4 Your own odometry [10 pts]

Maps to CLO 1 | **Bloom's Level** Apply

Context. The plugin's `/odom` cannot represent unequal wheels. Here you replace it with an odometry node of your own, built from the wheel joint angles.

TASKS

(a) **Write a node** `odometry_node` that subscribes to `/joint_states` and publishes `nav_msgs/Odometry` on `/odom_own` and the `odom` → `base_link` transform. It should:

- find the two wheel joints *by name*, and initialize on the first message;
- turn each change in wheel angle into a wheel travel with its own radius, $\Delta s_{L,R} = r_{L,R}\,\Delta\varphi_{L,R}$, then into Δs and $\Delta\theta$ using track width b; note that `/joint_states` reports the *absolute cumulative* joint angles, so you must store the previous angles to compute the change;
- integrate the pose with two different methods from Euler, Trapezoidal, RK2, Exact;
- have parameters `r_left`, `r_right`, `b`, `method`.

Launch the simulation with `plugin_tf:=false`, which will make the differential-drive plugin not publish the transform while your node publishes the transform.

(b) **Validate.** Set your node to your robot's *actual* dimensions from `pairs.yaml`, and repeat the open-loop square of Problem 2. Compare the end pose from `/odom_own`, `/odom` and `/ground_truth/odom`. Your node should beat the plugin; explain why.

(c) **Euler or exact?** On the square, the two methods barely differ. Drive an arc instead ($v = 0.2\,\mathrm{m/s}$, $\omega = 0.2\,\mathrm{rad/s}$ for $30\,\mathrm{s}$) and run both methods on the same data (record `/joint_states` with `ros2 bag` and replay it, or run two nodes at once with different output topics). Report the difference at the end, and discuss the implications.

Problem 5 Calibrate your robot: UMBmark [20 pts]

Maps to CLO 3 | **Bloom's Level** Analyze

Context. A new robot arrives with a datasheet: the `nameplate` block for each robot in `pairs.yaml`, one wheel radius r_n and a track width b_n. They are close to the truth, but not equal to it. UMBmark (Borenstein and Feng, 1996) finds out how wrong they are by driving in squares.

The provided `umbmark_driver.py` drives one square of side L, steering only by *your* odometry: it holds the odometry heading along each leg, stops when odometry says it has gone L, and turns until odometry says 90°. The robot therefore believes it has returned to its start. At the end the driver compares the true pose with the odometry pose and appends the error $(\varepsilon_x, \varepsilon_y, \varepsilon_\theta)$, true pose minus odometry, to a CSV file.

TASKS

(a) **Two kinds of error.** Assuming that nameplate values are being used in the odometry, but the robot still moves according to the actual physical properties in Gazebo, derive that:

- each 90° turn, as reported by odometry, really turns $\frac{\pi}{2}+\alpha$, with $\alpha = \frac{\pi}{2}\left(\frac{\bar{r}\,b_n}{r_n\,b} - 1\right)$, in *both* directions;
- each straight leg motion, although odometry says it is straight, really changes heading by $\beta = \dfrac{L\,(r_R - r_L)}{r_n\,b}$ (positive to the left), in both directions; you've already derived this earlier.

(b) **Where does the robot end up?** Treating α and β as small and adding the effect of each on the final position one at a time, show that

CCW: $\varepsilon_x \approx 2L(\alpha+\beta)$, $\varepsilon_y \approx -2L(\alpha+\beta)$; CW: $\varepsilon_x \approx 2L(\alpha-\beta)$, $\varepsilon_y \approx 2L(\alpha-\beta)$.

Explain why running in only one direction (CCW or CW) cannot separate α from β.

(c) **Run it.** Set your odometry node to the *nameplate* values. We're assuming that we don't know the real radii and will see if the algorithm can determine these systematic errors. Run five squares clockwise and five counter-clockwise, relaunching the simulation before each:

```
ros2 run a2_description umbmark_driver.py --ros-args -p use_sim_time:=true \
    -p odom_topic:=/odom_own -p direction:=cw -p side:=2.0 -p csv:=umbmark.
    csv
```

Plot the ten end points and the two centres of gravity, and compare with your prediction.

(d) **Identify.** From the two centres of gravity, solve for α and β (use both ε_x and ε_y). Then derive corrected odometry parameters: a track width b' that makes the turns right, and a wheel-diameter ratio $E_d = r_R/r_L$ that makes the legs straight (keep the mean radius at r_n for now).

(e) **What UMBmark cannot see.** Compare b' with the `actual` b. Show that UMBmark actually measures $b\,r_n/\bar{r}$, so any common error in the wheel radius is invisible to it. Design a short experiment that measures $\bar{r}/r_n$ (on a real robot you would use a tape measure; here `/ground_truth/odom` plays that role), run it, and correct both the radii and b'. Write down the nameplate, after UMBmark, after the scale test, and actual values of all parameters in a table.

(f) **Validate.** Run one square in each direction with the fully calibrated parameters and report the end errors. Discuss.

Problem 6 Where is the wall? [10 pts]

Maps to CLO 2 | **Bloom's Level** Apply

Context. Open loop, your robot could not even close a square. These next problems close the control loop: this is similar to the corridor-robot developed in the class.

GIVEN

- **The LiDAR** publishes 360 ranges on `/scan`, one per degree: `ranges[i]` is the range along the direction $i°$ counter-clockwise from the robot's forward axis, so `ranges[270]` looks straight right. Range noise is Gaussian with $\sigma_r = 0.01\,\text{m}$. Invalid returns are `inf`, i.e. if a LiDAR beam doesn't collide with an obstacle it returns `inf`.
- **Wall-following start:** `sim.launch.py ... x:=-4.0 y:=-5.2` puts the robot facing $+x$ with the south wall 0.8 m to its right. Ahead along that wall are a 1 m doorway (at x between 2 and 3) and, further round the arena, a buttress sticking out of the west wall.

To follow a wall on its right, the robot uses two beams, as shown in Figure 1.

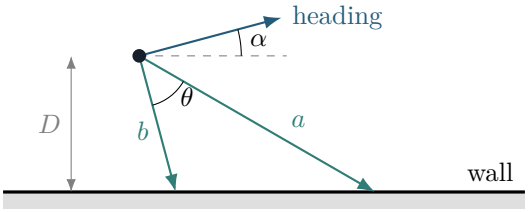


Figure 1: Two LiDAR beams used to estimate the robot's distance D and angle α to a wall on its right.

TASKS

(a) Let α be the angle between the robot's heading and the wall, positive when the robot points away from it, and D be the perpendicular distance from the LiDAR to the wall. Derive expressions for α and D in terms of the two ranges a and b and the angle θ between them. Specify the angles of the two beams that you'll use, measured conter-clockwise from the robot's forward axis, and explain why you chose them.

(b) Both ranges carry independent noise of standard deviation σ_r. Using first-order propagation of uncertainty from class, find σ_α when the robot runs parallel to the wall ($\alpha = 0$), as a function of θ and D. Evaluate it in degrees for $\sigma_r = 0.01\,\text{m}$, $D = 0.8\,\text{m}$ and $\theta = 15°, 30°, 45°, 60°, 75°$.

(c) Your result says: make θ as large as possible. Name two things the noise model leaves out that make a large θ a bad idea. In particular, how far ahead along the wall does beam a land, and what happens when the wall ends there (a doorway, or an outside corner)?

(d) Gazebo's LiDAR adds the same Gaussian noise to every beam, whatever its range or the angle at which it hits the wall. So the simulator will agree with (b). Is that a reason to trust (b)?

(e) Choose θ for Problem 7 and justify the choice in two or three sentences.

Problem 7 Follow the wall [20 pts]

Maps to CLO 3 | **Bloom's Level** Apply, Analyze

Context. You now use the two-beam estimate of Problem 6 to close the loop, starting from the wall-following pose given there.

TASKS

(a) **Write a node** `wall_follower` that, on every `/scan`, forms the error e, as defined by you, and then publishes a constant forward speed $v = 0.2\,\text{m/s}$ with turn rate

$$\omega = K_p e + K_d \dot{e} + K_i \int e\,dt, \qquad |\omega| \le 1.5\,\text{rad/s}.$$

Log time, D, α, e and ω to a CSV file. Start from the wall-following pose, as described in Problem 6/Given, so the robot begins 0.3 m too far from the wall.

(b) **P only.** Try at least three values of K_p. For each, report the overshoot, the time to settle within 2 cm of D_{ref}, and the remaining error, and plot D against time.

(c) **PD.** Add K_d and repeat (b). Use your result from Problem 6(b) to explain the noise in ω.

(d) **Is I needed?** Report the steady-state error of your best PD controller. Your robot's wheels are slightly unequal (Problem 2), which acts as a constant disturbance turning it one way. Estimate the steady-state error that disturbance should cause under your K_p, compare it with the LiDAR noise, and say whether an I term is worth having here.

(e) **Make it fail.** Let the robot run until it reaches the doorway (and, if it survives, on round the arena to the buttress). Describe what happens, and explain it using the geometry of Problem 6: which beam sees the gap first, and how far ahead? Propose one change that would fix it; you do not need to implement it.

Problem 8 Follow a path: pure pursuit [20 pts]

Maps to CLO 3 | **Bloom's Level** Apply, Analyze

Context. The square in Problem 2 was driven open loop, and the wall follower reacted to features the LiDAR could see. Pure pursuit follows a *given* path using the robot's own pose estimate. At every step it picks a *goal point* on the path a fixed lookahead distance L_d ahead of the robot, and steers along the circular arc that leaves the robot tangent to its heading and passes through that point (Figure 2).

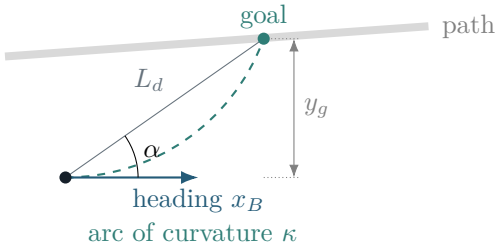


Figure 2: Pure pursuit: the goal point lies on the path at distance L_d; the robot steers along the arc (dashed) through it.

GIVEN

- **Paths,** both counter-clockwise and starting at the spawn pose (origin, facing $+x$): a *circle* of radius $R = 1.5\,\text{m}$ centred at $(0, 1.5)$, and a *square* of side $2\,\text{m}$ with corners $(0,0)$, $(2,0)$, $(2,2)$, $(0,2)$. Both fit well inside the arena.
- **Waypoints:** you do not generate the paths yourself. The package ships them as `paths/circle.csv` and `paths/square.csv`: points 1 cm apart, columns `x, y, yaw`, in the `odom` frame, with the last point repeating the first. Load one with

```
f = os.path.join(get_package_share_directory('a2_description'), 'paths', '
    square.csv')
path = np.loadtxt(f, delimiter=',', skiprows=1)     # N x 3: x, y, yaw
```

  To see the path in RViz, launch with `path:=circle` or `path:=square`.
- **Pose:** steer using `/odom`, as a real robot would. Use `/ground_truth/odom` only to evaluate.
- **Motion:** constant $v = 0.2\,\text{m/s}$, $|\omega| \leq 1.5\,\text{rad/s}$.

TASKS

(a) **Write a node** `pure_pursuit` with parameters `lookahead` L_d and v. It should load the waypoints from the CSV file (a parameter selects `circle` or `square`), and wait for `/odom` before moving. On every `/odom` message it should find the nearest point on the path (search forward only, so that it never jumps back across a corner), take as goal the first point further along the path at distance $\geq L_d$, transform it into the robot frame, and command (v, ω). Stop after one lap. Log time, the `/odom` pose, the true pose, the cross-track error of each, and ω.

(b) **Circle.** Run the circle with $L_d = 0.5\,\text{m}$, once with `pair:=REF` and once with your pair ID. For each, plot the reference circle, the `/odom` path and the true path on equal axes, and the cross-track error against time according to each. Report the steady-state error according to `/odom` and according to ground truth, and explain any difference.

(c) **Square.** On `REF`, run the square with $L_d = 0.3$, 0.6 and $1.0\,\text{m}$. Plot the three paths on one figure, and report the largest deviation from the path at the corners against L_d. Then try $L_d = 0.15\,\text{m}$ and describe what changes. Using the ω limit, find the L_d below which a corner demands more turn rate than the robot is allowed.

(d) **Closed loop is not enough.** Run the square with your own pair ID and $L_d = 0.3\,\text{m}$. How well does it close according to `/odom`, and according to ground truth? Compare the true end-point error with your open-loop square from Problem 2. Using Problem 3, explain why closing the loop removed one kind of error and not the other, and name two ways to remove the rest.

Extra Credit Questions

Problem 9 A wrong track width [15 bonus pts]

Maps to CLO 1, 3 | **Bloom's Level** Analyze

Context. Problem 3 found a fault caused by unequal wheels. This problem plants a second one on purpose: the plugin is told the wrong track width, while the physical robot is unchanged.

TASKS

(a) Assume that the robot is back at its starting position. The differential-drive plugin is now informed that the track width $b_p = s\,b$. You now send a pure rotation command, $v = 0$, and stop the robot when `/odom` says the robot has turned one full rotation. **Derive** the following from the kinematic model.

- How far does the robot *actually* turn?
- What does `/odom` report?
- Does the error grow, shrink or stay the same if you double the angular speed?
- What if the separation is 10% too *large*? Is the effect symmetric?

(b) **Write a node** `spin_test` (Python, `rclpy`) that

- waits until it has received both `/odom` and `/ground_truth/odom`;
- publishes `angular.z` $= \omega$ (a parameter) on `/cmd_vel` until the yaw, θ, from `/odom` has advanced by 2π, then publishes zero;
- waits 2 s for the robot to come to rest;
- prints the total yaw change according to `/odom`, according to `/ground_truth/odom`, and their ratio.

Note that yaw arrives as a quaternion; you can convert it to an angle using the formula, $\theta = 2\,\mathrm{atan2}(q_z, q_w)$. Both yaws wrap at $\pm\pi$, and a full turn crosses that point, so accumulate an *unwrapped* angle.

(c) **Run** three trials at each of $\omega = 0.5$ and 1.0 rad/s, with `plugin_separation_scale` set to 0.9 and then 1.1: twelve runs. Tabulate the results.

You'll have to relaunch with the plugin's wheel separation set **10% too small**:

```
ros2 launch a2_description sim.launch.py pair:=P03 plugin_separation_scale
    :=0.9
```

(d) **Compare** with your derivation. Why is the *ratio* a better quantity to report than the true yaw change alone? Does your data show any asymmetry between 0.9 and 1.1, or any dependence on ω? If so, what physical effect could cause it, given that the kinematic model predicts none?

Problem 10 Follow a moving target [15 bonus pts]

Maps to CLO 3 | **Bloom's Level** Apply

Context. A wall stays put. This problem closes the loop on something that moves.

GIVEN

The target: `sim.launch.py ... cone:=true` adds an orange traffic cone 1.5 m ahead of the robot. It moves only when told to: `ros2 run a2_description cone_driver.py` drives it along a scripted path (`profile:=line`, `stopgo` or `arc`; `speed` in m/s), and you can steer it by hand with teleop remapped to `/cone/cmd_vel`. Its true pose is on `/cone/ground_truth/odom`.

TASKS

(a) **Write a node** `cone_follower` that, on every `/scan`, finds the nearest cone and moves towards it with a proportional controller:$0 \le v \le 0.5\,\text{m/s}$ and $|\omega| \le 1.5\,\text{rad/s}$. If nothing is detected, stop. Log ρ, φ, v and ω.

(b) **Measure.** Run `profile:=line` at 0.2 m/s with two values of the proportional gain, relaunching each time. From your log, report the steady-state error. Check it against ground truth as well (`/ground_truth/odom` and `/cone/ground_truth/odom`); what offset do you have to account for, and why?

(c) **Why not a huge gain?** Give two reasons, from your data or from the sensing, why making the gain large is not the fix. Then propose a change to the controller that removes the steady-state gap without raising proportional gain, and explain why it works.

(d) Run `profile:=stopgo` and `profile:=arc`. Describe one behaviour of your follower on each that the steady-state analysis does not predict.

References